

Learning to Walk with Hybrid Serial-Parallel Linkages: a Case Study on the Kangaroo Robot

Fabio Amadio¹, Hongbo Li², Lorenzo Uttini², Dimitrios Kanoulas², Serena Ivaldi¹, Valerio Modugno², and Enrico Mingo Hoffman¹

¹ Université de Lorraine, CNRS, Inria HUCEBOT Team, LORIA, France
{fabio.amadio, serena.ivaldi, enrico.mingo-hoffman}@inria.fr

² University College London, London, UK
{hongbo.li.24, lorenzo.uttini.24, d.kanoulas, v.modugno}@ucl.ac.uk

Abstract. Humanoid robots increasingly adopt hybrid serial-parallel kinematics to improve structural stiffness, mass distribution, and impact robustness. However, these mechanisms introduce complexity associated with simulation and control, which affects algorithms for Reinforcement Learning (RL)-based locomotion. This paper presents a case study of an RL end-to-end pipeline that trains walking policies for Kangaroo, a 72 degrees of freedom biped whose legs contain several hybrid serial-parallel chains, without kinematic simplifications. Training is performed using the *Isaac Lab* framework, leveraging the *Isaac Sim* built-in constraint capabilities. We perform an ablation study on the observation state to assess the role of redundant information from Kangaroo’s measured state (passive/active joint measurements). Furthermore, a set of trained policies is validated in *MuJoCo*, demonstrating a degree of robustness to the *Sim-to-Sim* gap. The closed-loop behaviors of the tested policies successfully transfer in most cases, despite differences in how contacts and constraints are modeled across the two simulators. Finally, we analyzed how minor differences in the action rate penalty weight can deeply affect the locomotion stability of the resulting policies when deployed in a different simulator.

Keywords: AI & ML & Deep RL · Dynamics and Control

1 Introduction

In recent years, humanoid bipedal robots have moved beyond laboratory settings into real-world environments. Significant advancements in control systems and mechatronic design have enabled this transition. In particular, Reinforcement Learning (RL) has become essential for achieving robust locomotion in unstructured environments [5,6], while, on the mechatronics side, there has been a growing adoption of hybrid serial-parallel linkages in robot design.

These mechanisms enable higher structural stiffness and improved dynamic performance by relocating heavy components, such as actuators, closer to the robot’s base, thereby reducing the inertia during impacts that happen during locomotion. Such structures can be observed in most modern humanoid robots, including Cassie [12] and PAL Robotics Kangaroo [3], to name a few.

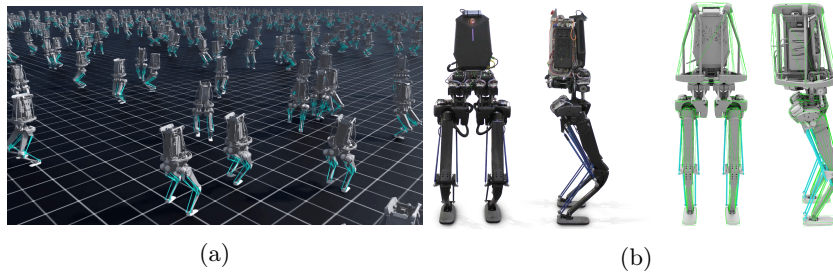


Fig. 1: (a) Multiple Kangaroo robots simulated in *Isaac Sim*. (b) Kangaroo robot and its model in *Isaac Sim* (collision meshes in green).

In recent years, the robotics humanoid community started to show interest in the explicit modeling of hybrid serial-parallel kinematic chains present in bipedal systems for both simulation and control [1–3]. However, the modeling and control of such mechanisms still remain a challenge due to their inherent complexity. To cope with this, modeling is often simplified, ignoring specific constraints and dynamic behaviors of the system [7], but with the risk of obtaining suboptimal control strategies or inaccurate simulations.

In the context of RL, the possibility of explicitly including hybrid serial-parallel mechanisms is strictly related to the simulation tools used for training. Currently, both *Isaac Sim*³ and *MuJoCo* [11] support hybrid serial-parallel kinematic chains. Nevertheless, RL-based solutions for robots with hybrid serial-parallel linkages remain uncommon. Moreover, it is uncommon for control policies to be trained directly in motor space (the space of active joints) [13]; instead, simplified models are often used [5], and their outputs are subsequently transformed into motor space through dedicated routines that are not considered during training [10].

Motivated by these challenges, this paper presents a complete simulation and learning pipeline developed for PAL Robotics Kangaroo, a bipedal humanoid robot incorporating multiple hybrid serial-parallel mechanisms in its leg design, posing non-trivial challenges for control and policy learning. Our main contributions are:

1. **Training pipeline:** The Kangaroo robot is modeled within NVIDIA *Isaac Sim* (Fig. 1a), while the policy is trained using Proximal Policy Optimization (PPO) [8] employing the *Isaac Lab* framework⁴ [4]. Code available at: https://github.com/hucebot/isaacsim_kangaroo.
2. **Sim-to-Sim validation:** To evaluate robustness and transferability, we validate the trained policies by deploying them in *MuJoCo*. Code available at: https://github.com/hucebot/mujoco_kangaroo_sim2sim.
3. **Ablation studies:**
 - *Observation space composition.* We analyze the impact of including the state of different joints in the observation, an especially relevant factor for systems with complex joint structures like Kangaroo.

³ NVIDIA *Isaac Sim*: <https://developer.nvidia.com/isaac/sim>

⁴ NVIDIA *Isaac Lab*: <https://isaac-sim.github.io/IsaacLab>

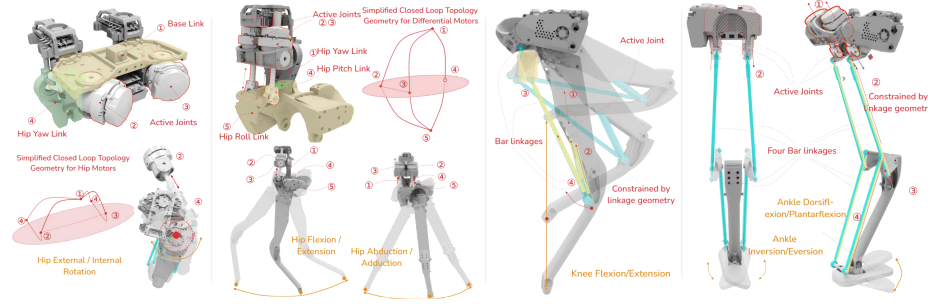


Fig. 2: Kangaroo kinematic groups, from left to right: *Hip External/Internal Rotation*, *Hip Flexion/Extension & Abduction/Adduction*, *Knee Flexion/Extension*, and *Ankle Dorsiflexion/Plantarflexion & Inversion/Eversion*.

- *Action rate penalty sensitivity*. We show that small variations in action rate (from a lower associated penalty weight) can drastically affect deployment success in *MuJoCo*, despite having no impact in *Isaac Sim*.

The video at <https://youtu.be/sBrSp9quXDY> shows the motion of Kangaroo’s closed sub-mechanisms, together with training and *Sim-to-Sim* results.

2 Modeling the Kangaroo Robot

The full model of Kangaroo includes a total of 72 degrees of freedom (DOFs), comprising 60 rotational passive and 12 linear active DOFs. Each leg of Kangaroo is composed of four kinematic groups (see Fig. 2), as described in [3]:

- 1 DOF *Hip External/Internal Rotation*, actuated by **Motor 1** (M1);
- 2 DOFs *Hip Flexion/Extension & Abduction/Adduction*, actuated by **Motors 2 and 3** (M2, M3);
- 1 DOF *Knee Flexion/Extension*, actuated by the **Leg Length Motor** (LLM);
- 2 DOFs *Ankle Dorsiflexion/Plantarflexion & Inversion/Eversion*, actuated by **Motors 4 and 5** (M4, M5).

It is worth noting that below the knee, no electronics are present, making Kangaroo a robust platform for locomotion and agile actions. From a sensing perspective, Kangaroo is equipped with an IMU located at the waist, 10 rotational absolute encoders (5 per leg) placed on selected passive joints, and 2 linear absolute encoders (1 per leg) installed on the LLM actuators (see Fig. 2).

In the real Kangaroo, because not all linear actuators are equipped with absolute encoders, a calibration procedure is required each time the robot is started. By leveraging the known kinematics of the closed chains and the readings from the absolute rotational encoders, it is possible to initialize the positions of the linear actuators. This enables position control in addition to velocity and force control. Table 1 presents the actuator limits for each closed sub-mechanism.

Table 1: Actuator limits and starting configurations for each closed sub-mechanism in Kangaroo’s lower body.

<i>Sub-mech.</i>	<i>Min [m]</i>	<i>Max [m]</i>	<i>Start [m]</i>
hip yaw	-0.02	0.02	0.0
hip roll/pitch	-0.04	0.04	0.0
knee	0.00	0.15	0.0375
ankle roll/pitch	-0.02	0.02	0.0

Table 2: Actuators gains.

<i>Motor</i>	<i>K_p [N/m]</i>	<i>K_v [N·s/m]</i>
M1	$3 \cdot 10^5$	1095
M2	$7 \cdot 10^5$	1673
M3	$7 \cdot 10^5$	1673
M4	$8 \cdot 10^5$	1789
M5	$8 \cdot 10^5$	1789
LLM	$10 \cdot 10^5$	2000

2.1 Simulation

The hybrid serial-parallel closed linkages are modeled as kinematic chains that are “opened” at specific passive joints, with constraints applied to enforce closure. In the case of the Kangaroo, a total of 60 DOFs, corresponding to the passive joints, must be constrained to accurately simulate the closed-chain behavior. The implementation of these constraints varies depending on the simulator.

Isaac Sim permits the modeling of hybrid serial-parallel kinematic chains by the inclusion of passive joints to *close* the open chains. In particular, we started from a URDF description of Kangaroo, which includes all the DOFs but the ones that close the chains. We imported it into the *Isaac Sim*, and we included manually, through the GUI, the missing passive joints in the model (described using the Universal Scene Description (USD) format⁵), thereby enabling the representation of closed kinematic chains. *Isaac Sim* permits also using *articulation* to achieve better performance. However, *articulations* do not support the direct representation of closed-loop kinematic structures. To address this limitation, one joint was excluded from each closed kinematic chain in the *articulation* hierarchy. Despite removing them from the *articulation*, the passive joints are still present in the robot description, ensuring closure in the hybrid serial-parallel mechanisms. Actuated joints are controlled through simulated position controllers defined by Table 2 parameters, and a simulation time step of 0.005 s has been used. These values have been tuned by hand to obtain an effective reference tracking for the actuated linear joints and overall stability of the simulated closed chains.

MuJoCo supports closed loops by enabling explicit constraints between different bodies. Hence, Kangaroo’s closed kinematic chains have been implemented using a series of **equality connect** constraints for each kinematic group of both legs. Proper tuning of different simulation parameters has been fundamental to maintain the constraints and the overall stability of the *MuJoCo* simulation. Notably, the solver parameters for equality constraint simulation, i.e., the **solref**, comprising two values: **timeconst** and **damping_ratio**, representing the time constant and damping ratio of the virtual spring-damper system enforcing the constraint, respectively. We found that with **time_constant** value of 0.005 and a **damping_ratio** of 1.0 (critically damped), we were able to simulate the Kangaroo system maintaining the integrity of the closed-chain structure. Regarding

⁵ USD: <https://openusd.org>

the actuation, also in *MuJoCo* we adopted the same position control parameters of Table 2, empirically verifying the achievement of a valid reference tracking. To increase numerical stability, we set the simulation time step to 0.001 s , a smaller value w.r.t. the time step used in *Isaac Sim*.

Finally, collisions were modeled using convex hulls of selected links, without including all the links forming each sub-mechanism to avoid internal collisions that could produce undesired internal forces (Fig. 1b).

3 Method

The locomotion control problem is formulated as a classical RL problem where an agent, at each time step t , receives an observation $\mathbf{o}_t$ from the environment, selects a control action $\mathbf{a}_t$, and collects a reward r_t . Control actions are sampled from a stochastic policy $\mathbf{a}_t \sim \pi(\mathbf{a}_t|\mathbf{o}_t)$ at a frequency of 50 Hz . In the considered locomotion setting, the primary objective is to find a policy able to minimize the difference between the robot’s actual base velocity and a commanded base velocity. For simplicity, we are considering only the two linear components along the x and y axes in the gravity-aligned robot frame, denoted as $\bar{\mathbf{v}}^{xy}$, and the angular velocity around the world frame z axis (yaw), denoted as $\bar{\phi}$. We simulate a Kangaroo robot standing on a flat terrain, initialized with the joints configured in the actuators’ starting positions reported in Table 1. The duration of each episode is 20 s , at the beginning of which, a random velocity command is sampled from uniform distributions for the robot to follow. Linear velocity references along the x axis are sampled from $\mathcal{U}(0,1)\text{ [m/s]}$, while references along the y axis from $\mathcal{U}(-1,1)\text{ [m/s]}$ (where $\mathcal{U}(a,b)$ denotes the continuous uniform probability distribution between the minimum a and maximum b). Angular velocity references are instead updated at each policy update steps according to the current orientation error w.r.t. to a target heading angle sampled from $\mathcal{U}(-\pi,\pi)\text{ [rad]}$, and clipped between the min-max interval $(-1,1)\text{ [rad/s]}$. The episode terminates in case the robot falls, when its base link height goes below 0.6 m .

We use the PPO algorithm to train the policy (adopting the *RSL RL* implementation⁶), leveraging multiple parallel *Isaac Sim* simulations and GPU acceleration provided by the *Isaac Lab* framework. In the following, we provide a more detailed description of the action and observation spaces, the rewards, and the domain randomization settings.

3.1 Observation Space

We are considering the following components to define the observation $\mathbf{o}_t$:

- $\mathbf{v}_t \in \mathbb{R}^3$, base linear velocity (in body frame);
- $\boldsymbol{\omega}_t \in \mathbb{R}^3$, base angular velocity (in body frame);
- $\tilde{\mathbf{g}}_t \in \mathbb{R}^3$, gravity direction projected on the body frame;
- $\bar{\mathbf{v}}_t^{xy} \in \mathbb{R}^2$, base linear velocity command (xy axes);

⁶ *RSL RL*: https://github.com/leggedrobotics/rs1_rl

- $\bar{\phi}_t \in \mathbb{R}$, base angular velocity command (yaw);
- $\mathbf{q}_t^p \in \mathbb{R}^{10}$, position of the measurable passive joints;
- $\dot{\mathbf{q}}_t^p \in \mathbb{R}^{10}$, velocity of the measurable passive joints;
- $\mathbf{q}_t^{ll} \in \mathbb{R}^2$, position of the LLM;
- $\dot{\mathbf{q}}_t^{ll} \in \mathbb{R}^2$, velocity of the LLM;
- $\mathbf{q}_t^m \in \mathbb{R}^{10}$, position of M1, M2, M3, M4, M5;
- $\dot{\mathbf{q}}_t^m \in \mathbb{R}^{10}$, velocity of M1, M2, M3, M4, M5;
- $\mathbf{a}_{t-1} \in \mathbb{R}^{12}$, actions applied at the previous step.

It is worth noticing that the full joint state of Kangaroo is $\mathbf{q} \in \mathbb{R}^{72}$, which encompasses both measurable and non-measurable, passive and active joints. As previously mentioned in Sec. 2, in the real system, positions of $\mathbf{q}^{ll}$ and $\mathbf{q}^p$ can be measured directly via absolute encoders, instead positions of $\mathbf{q}^m$ is obtained by integration starting from a calibrated initial position. Given this distinctive characteristic of our system, in Sec. 4.1, we analyze the effects of composing the observation vector using a diverse set of joints. In particular, we will consider three different structures for the observation vector $\mathbf{o}_t$:

- *Full*: $\left[\mathbf{o}_t^c, \mathbf{q}_t^{pT}, \dot{\mathbf{q}}_t^{pT}, \mathbf{q}_t^{llT}, \dot{\mathbf{q}}_t^{llT}, \mathbf{q}_t^{mT}, \dot{\mathbf{q}}_t^{mT} \right]^T \in \mathbb{R}^{68}$
- *Measured*: $\left[\mathbf{o}_t^c, \mathbf{q}_t^{pT}, \dot{\mathbf{q}}_t^{pT}, \mathbf{q}_t^{llT}, \dot{\mathbf{q}}_t^{llT} \right]^T \in \mathbb{R}^{48}$
- *Actuated*: $\left[\mathbf{o}_t^c, \mathbf{q}_t^{llT}, \dot{\mathbf{q}}_t^{llT}, \mathbf{q}_t^{mT}, \dot{\mathbf{q}}_t^{mT} \right]^T \in \mathbb{R}^{48}$

where $\mathbf{o}_t^c = \left[\mathbf{v}_t^T, \boldsymbol{\omega}_t^T, \tilde{\mathbf{g}}_t^T, \bar{\mathbf{v}}_t^{xyT}, \bar{\phi}_t, \mathbf{a}_{t-1}^T \right]^T$ contains the state of the floating base, the velocity commands, and the action applied at the previous step.

3.2 Action Space

The action space of the bipedal robot is defined as the set of target positions for all actuated joints on each leg. As previously shown, each leg is equipped with six independently controlled linear actuators, resulting in a total of 12 continuous action dimensions. These actuated joints correspond to functionally distinct DOFs, each enabling a specific motion critical for dynamic locomotion. Hence, the action vector $\mathbf{a} \in \mathbb{R}^{12}$ describes the position of the linear actuators. The ranges of allowed motor positions are restricted to a tight interval due to the linear nature of Kangaroo’s actuators. If not carefully taken into consideration, this characteristic can hinder the learning process of the PPO agent, which could be unable to correctly explore the allowed action space, for instance, because of a too high initial policy’s standard deviation parameter. In this work, we decided to post-process the raw actions $\mathbf{a}_t$ sampled from the policy distribution to consistently obtain valid references for the actuated joints $\hat{\mathbf{q}}_t^m$. The procedure involves two steps: (i) we scale the raw actions into the range $(-1,1)$ using a *tanh* function, (ii) re-scale each action’s dimension from $(-1,1)$ to the actual joint limits. Detailed expressions are reported below.

$$\begin{aligned} \tilde{\mathbf{a}}_t &= \tanh(0.1 \cdot \mathbf{a}_t + \delta) \\ \hat{\mathbf{q}}_t^m &= \tilde{\mathbf{a}}_t \cdot (\bar{\mathbf{q}}^m - \underline{\mathbf{q}}^m)/2 + (\bar{\mathbf{q}}^m + \underline{\mathbf{q}}^m)/2 \end{aligned}$$

$\underline{q}^m, \bar{q}^m \in \mathbb{R}^{12}$ are the lower and upper bounds of the actuated joints with, while $\delta \in \mathbb{R}^{12}$ is a constant offset vector, which is assigned with the only non-zero values (-0.54) at the two dimensions associated to each leg's LLM to map $a=0$ to their corresponding starting position $\hat{q}^m = 0.0375 m$.

3.3 Rewards and Penalties

The total reward considered during the training comprises different terms, with both rewards (positive) and penalties (negative), that are weighted and summed together. Let us first define the following quantities:

- *Air times* ${}^{L/R}T_t^{air}$, the time that the left (right) foot is spending in the air since the last lift off (0 if in contact);
- *Last air times* ${}^{L/R}\bar{T}_t^{air}$, the time the left (right) foot spent in the air before the last contact;
- *Contact times* ${}^{L/R}T_t^{con}$, the time the left (right) foot is spending in contact since the last contact (0 if not in contact);
- *Last contact times* ${}^{L/R}\bar{T}_t^{con}$, the time the left (right) foot spent in contact before the last detach.

Additionally, we will refer to the robot angular velocity around the world frame z axis (yaw) as ϕ_t , and to the velocity of the feet links expressed in world frame as ${}^{L/R}\mathbf{v}_t$. Finally, we are using x , y , and z superscripts to denote the components of vectors $\mathbf{v}_t$ and $\boldsymbol{\omega}_t$ relative to the axis x , y , and z , respectively. All the reward and penalty terms used during the policy training are reported in Table 3. Moreover, a fixed termination penalty of -200 is assigned in case the robot falls.

Table 3: Rewards and penalties.

Name	Expression	Weight
lin vel xy	$\exp(-\ \mathbf{v}_t^{xy} - \mathbf{v}_t^{xy}\ ^2 / 0.25)$	2.0
ang vel z	$\exp(-(\phi_t - \phi_t)^2 / 0.25)$	1.0
feet air	$\begin{cases} {}^L T_t^{air} & \text{if } {}^R T_t^{air} = 0 \\ {}^R T_t^{air} & \text{if } {}^L T_t^{air} = 0 \\ 0.0 & \text{otherwise} \end{cases}$	2.0
lin vel z	$\ \mathbf{v}_t^z\ ^2$	-0.2
ang vel xy	$\ \boldsymbol{\omega}_t^{xy}\ ^2$	-0.05
flat orient.	$\ \tilde{\mathbf{g}}_t^{xy}\ ^2$	-2.0
action rate	$\ \mathbf{a}_t - \mathbf{a}_{t-1}\ ^2$	-0.01
feet slide	$\sum_{j=L,R} \ \mathbf{v}_t^j\ \cdot [{}^j T_t^{con} > 0]$	-0.25
diff. steps	$ {}^L \bar{T}_t^{air} - {}^R \bar{T}_t^{air} + {}^L \bar{T}_t^{con} - {}^R \bar{T}_t^{con} $	-0.5

Table 4: Noise and domain randomization parameters.

Param	Range
noise $\mathbf{v}$	$[-0.1, 0.1]$
noise $\boldsymbol{\omega}$	$[-0.2, 0.2]$
noise $\tilde{\mathbf{g}}$	$[-0.05, 0.05]$
noise $\mathbf{q}^p$	$[-0.01, 0.01]$
noise $\dot{\mathbf{q}}^p$	$[-0.1, 0.1]$
noise $\mathbf{q}^{ll}$	$[-2.5, 2.5] \cdot 1e-3$
noise $\dot{\mathbf{q}}^{ll}$	$[-2.5, 2.5] \cdot 1e-2$
noise $\mathbf{q}^m$	$[-2.5, 2.5] \cdot 1e-3$
noise $\dot{\mathbf{q}}^m$	$[-2.5, 2.5] \cdot 1e-2$
base xy	$[-0.5, 0.5]$
base yaw	$[-3.14, 3.14]$
stat. fr.	$[0.7, 1.0]$
dyn. fr.	$[0.4, 0.7]$

3.4 Domain Randomization

Observation vectors $\mathbf{o}_t$ are corrupted by an additive noise sampled from different uniform distributions for each component taken on the robot, to mimic the presence of measurement noise. Moreover, static and dynamic friction coefficients are randomized together with the robot’s initial pose. Table 4 provides details of all the measurement noises and domain randomization terms.

4 Policy Training

Training with PPO was carried out using 2048 parallel environments and the default hyperparameters provided by *Isaac Lab*. Both the actor and critic networks featured hidden layers of dimensions [256, 256, 128], with Exponential Linear Unit (ELU) activation functions. We maintained the self-collisions enabled in the simulator throughout all the training and evaluation experiments. We found that 2000 learning iterations (98.3M environment steps) were enough to converge to a walking policy. Fig. 3a shows an example walk obtained at the end of the training process. Training has been carried out on a *NVIDIA RTX 2000 Ada GPU*, taking approximately 2 hours to complete a policy training, despite the need to simulate the closed kinematic chains and the enabled self-collisions.

Given the complex structure of the robot’s kinematics and the number of DOFs, we decide to focus our analysis on understanding which kind of joint measurements are decisive for a successful locomotion policy training. In the following, we present an ablation study in which we compare training performance obtained with the three different observation structures proposed in Sec. 3.1.

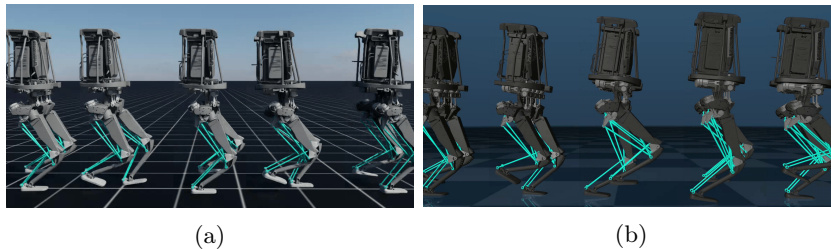


Fig. 3: (a) Time-lapse of one of the obtained walking policies running on *Isaac Sim*. (b) Time-lapse of one *Sim-to-Sim* policy deployment in *MuJoCo*.

4.1 Ablation Study: Changing joint state in the observations

For each observation vector structure, *Full*, *Measured*, and *Actuated*, we conducted 10 independent training sessions, each with a different random seed. Fig. 4 presents the comparison across these settings in terms of mean reward, along with the most

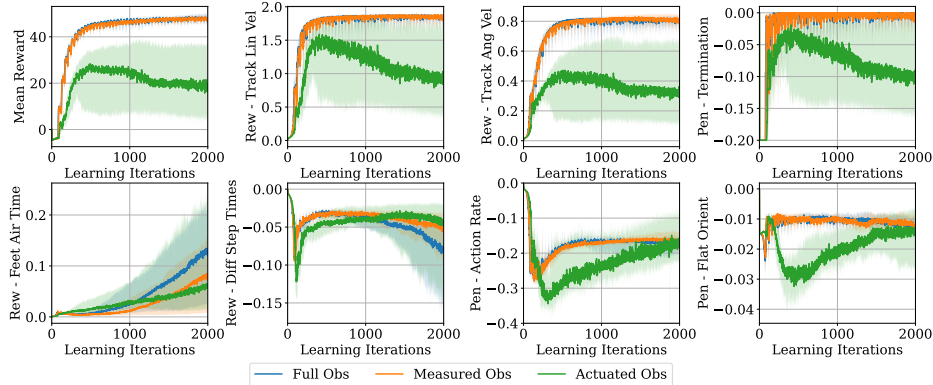


Fig. 4: Medians and 5th–95th percentile ranges of the mean total reward and some specific terms observed with the *Full*, *Measured*, or *Actuated* observations.

significant differences observed in specific reward and penalty terms. Results are shown using median values and 5th–95th percentile ranges. Policies trained with the *Actuated* observation vector consistently underperform in tracking accuracy compared to those using either the *Full* or *Measured* vectors. Moreover, they lead to more frequent robot falls: while early termination penalties largely disappear after a few iterations in the *Full* and *Measured* cases, they remain prominent with the *Actuated* configuration. Additional differences emerge in the action rate and *flat orientation* penalties. Policies based on the *Actuated* vector exhibit lower values and higher variability, particularly during early training, suggesting increased instability and difficulty in maintaining balance. Interestingly, the performance of policies trained with the *Full* and *Measured* observation vectors is largely comparable. In contrast, relying solely on motor states appears insufficient for successful training in this setup. This might possibly be attributed to the limited position range of actuators M1 to M5 (from $-0.02m$ to $0.02m$), especially when combined with additive uniform noise, which may diminish the relative informativeness of these inputs. Finally, an analysis of the remaining reward and penalty components highlights notable variability in both *feet air time* and *step difference* terms across all observation sets. This variability leads to a diversity of gait patterns, ranging from longer to shorter steps, and in some cases, asymmetric walking between the left and right legs. A more refined reward shaping strategy, along with careful tuning of individual reward components, may help future training runs converge toward more consistent and stable locomotion behaviors.

5 Sim-to-Sim Validation

To validate the effectiveness of the trained policies, we deployed them into a different simulation environment, namely *MuJoCo*. The *Sim-to-Sim* process poses significant challenges, due to the differences present in the two simulators, in particular in handling constraints and contacts.

When deploying the trained policies in *MuJoCo*, we replicated the same observation collection and action scaling, as described in Sec. 3.1 and Sec. 3.2, keeping the same policy update frequency of 50 Hz . For brevity, we deployed only policies that use the *Full* observation vector (Fig. 3b shows an example walk recorded in *MuJoCo*). To better cope with the reduced simulation time steps length and avoid major discontinuities in control inputs, we employed a first-order hold (FOH) function that interpolates between the previous and current policy actions across the simulation steps. This ensured smoothness and better joint reference tracking.

To evaluate *Sim-to-Sim* performance, we tested 10 different policies, each trained with a different random seed, in both *Isaac Sim* and *MuJoCo*. For each policy, we recorded the success rate, i.e., whether the robot fell, as well as the linear and angular tracking errors over 10 runs, each following a random command sampled in the same way as during training. Results are reported in Fig. 5 using box plots (refer to the *Default Reward* columns). Mann-Whitney U-test has been performed to assess the statistical significance of the differences observed in the collected results. While falling is always avoided when running the policies in the original *Isaac Sim* simulator, in *MuJoCo* the robot occasionally falls. The majority of the policies show a success rate between 70% and 90% (average 73%). However, tracking error shows a significant increase in the *Sim-to-Sim* deployment. If in *Isaac Sim* the mean linear and angular tracking errors are 0.095 m/s and 0.115 rad/s ; the policies deployed in *MuJoCo* achieve errors of 0.21 m/s and 1.56 rad/s . One of the factors that could explain such a worsening in performance is the different way the two simulators model contacts. Another relevant factor might be the effectiveness in enforcing closed-chain constraints. In fact, in *MuJoCo* closed-chain constraint violation errors are consistently higher than in *Isaac Sim*, with an average increase around 67%. Despite the inherent challenges of transferring policies across simulators, especially for complex closed-chain systems, it is encouraging to observe that most of the policies were still able to successfully stabilize the robot and achieve locomotion in *MuJoCo*, demonstrating a degree of robustness to the simulation gap.

5.1 Ablation Study: Training with a lower action rate weight

During the initial attempts at *Sim-to-Sim* deployment, we observed a drastic improvement when we started to use a action rate penalty weight of -0.01 . In fact, when using a lower weight of -0.005 (half of the original), we were experiencing great difficulties in keeping the balance. For this reason, we deployed another set of 8 policies trained using an action rate penalty weight of -0.005 (with all other terms identical to those in Table 3), and compared the results obtained over 10 runs for each policy. The use of a lower penalty weight resulted in an average increase of the policies' action rate of 29.52%. In Fig. 5, we also report the results obtained with this new reward (refer to the *Low act. rate weight Reward* columns).

In comparison with the results obtained by training with the default reward, it is possible to observe that lowering the action rate penalty weight did not result in major differences in *Isaac Sim* performance. Instead, in *MuJoCo*, it emerges a significant drop in the success rate, where we reach an average of only 27.5%, and

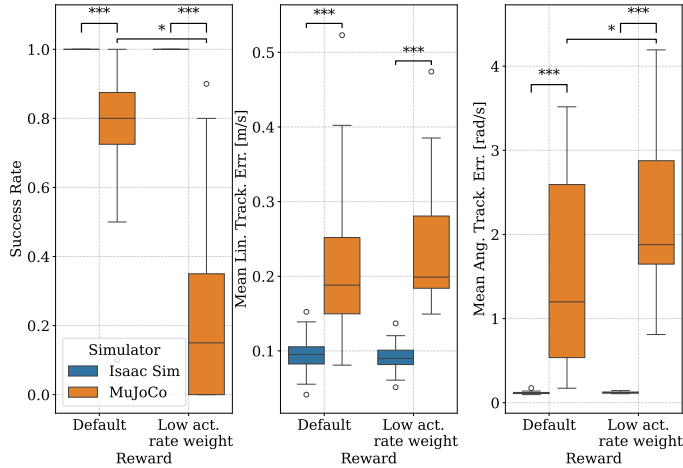


Fig. 5: Box plots of the success rates, linear and angular tracking errors from *Isaac Sim* and *MuJoCo*. 10 policies are trained with the reward of Table 3 (columns *Default*) and 8 trained with a lower action rate penalty weight of -0.005 (columns *Low act. rate weight*). Statistical significance is marked with either * (p-value < 0.05), ** (p-value < 0.01), or *** (p-value < 0.001).

an increase in the angular tracking error. These findings confirm the importance of carefully tuning action rate penalties to obtain transferable policies.

6 Conclusion and Future

In this work, we presented a case study on a locomotion training pipeline for Kangaroo, a bipedal humanoid robot featuring hybrid serial-parallel kinematic chains, based on *Isaac Sim* and successively transferred to *MuJoCo*. Unlike simplified approaches that train on reduced models requiring additional mappings, we chose to model the full complexity of Kangaroo and investigate whether it is possible to train and evaluate a walking policy directly at the motor level using existing tools for RL and simulation. Results highlight the importance of observation design according to the type of motors, as well as the presence of redundancy in the measurements. We conducted an extensive *Sim-to-Sim* validation process, deploying trained policies in *MuJoCo* achieving in most cases a stable walk despite the presence of a significant *Sim-to-Sim* gap. As next steps, we aim to refine the locomotion, possibly taking into account imitation learning to achieve more natural gaits [9] and deploy the trained policies on the physical Kangaroo robot, thus investigating *Sim-to-Real* challenges specific to systems with hybrid closed-chain mechanisms.

Acknowledgments. This work was supported by the French National Research Agency (ANR) under the project ANR-24-CE33-0753-01 (MeRLin), by the France 2030 program through projects PEPR O2R AS3 and PI3 (ANR-22-EXOD-007, ANR-22-EXOD-004),

the EU Horizon project euROBIN (GA n.101070596), by the UKRI FLF [MR/V025333/1] (RoboHike), and by the INRIA-UCL Associated Team LEG-AI. For the purpose of Open Access, the author has applied a CC BY public copyright license to any Author Accepted Manuscript version arising from this submission.

References

1. Batto, V., Flayols, T., Mansard, N., Vulliez, M.: Comparative metrics of advanced serial/parallel biped design and characterization of the main contemporary architectures. In: IEEE-RAS International Conference on Humanoid Robots (Humanoids). pp. 1–7 (2023)
2. Boukheddimi, M., Kumar, R., Kumar, S., Carpentier, J., Kirchner, F.: Investigations into exploiting the full capabilities of a series-parallel hybrid humanoid using whole body trajectory optimization. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). pp. 10433–10439 (2023)
3. Hoffman, E.M., Curti, A., Miguel, N., Kothakota, S.K., Molina, A., Roig, A., Marchionni, L.: Modeling and numerical analysis of kangaroo lower body based on constrained dynamics of hybrid serial-parallel floating-base systems. *Robotics and Autonomous Systems* **182**, 104827 (2024)
4. Mittal, M., Yu, C., Yu, Q., Liu, J., Rudin, N., Hoeller, D., Yuan, J.L., Singh, R., Guo, Y., Mazhar, H., Mandelkar, A., Babich, B., State, G., Hutter, M., Garg, A.: Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters* **8**(6), 3740–3747 (2023)
5. Radosavovic, I., Xiao, T., Zhang, B., Darrell, T., Malik, J., Sreenath, K.: Real-world humanoid locomotion with reinforcement learning. *Science Robotics* **9**(89) (2024)
6. Rudin, N., Hoeller, D., Reist, P., Hutter, M.: Learning to walk in minutes using massively parallel deep reinforcement learning. In: Conference on Robot Learning. pp. 91–100. PMLR (2022)
7. Ruscelli, F., Laurenzi, A., Mingo Hoffman, E., Tsagarakis, N.G.: A fail-safe semi-centralized impedance controller: Validation on a parallel kinematics ankle. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). pp. 1–9 (2018)
8. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. *CoRR* (2017)
9. Tang, A., Hiraoka, T., Hiraoka, N., Shi, F., Kawaharazuka, K., Kojima, K., Okada, K., Inaba, M.: Humanmimic: Learning natural locomotion and transitions for humanoid robot via wasserstein adversarial imitation. In: 2024 IEEE International Conference on Robotics and Automation (ICRA). pp. 13107–13114. IEEE (2024)
10. Tang, L., Liang, D., Gao, G., Wang, X., Xie, A.: Modeling and reinforcement learning-based locomotion control for a humanoid robot with kinematic loop closures. *Multibody System Dynamics* pp. 1–27 (2024)
11. Todorov, E., Erez, T., Tassa, Y.: Mujoco: A physics engine for model-based control. In: IEEE/RSJ International Conference on Intelligent Robots and Systems. pp. 5026–5033 (2012)
12. Xie, Z., Clary, P., Dao, J., Morais, P., Hurst, J., Panne, M.: Learning locomotion skills for cassie: Iterative design and sim-to-real. In: Conference on Robot Learning. pp. 317–329. PMLR (2020)
13. Zhang, Q., Han, G., Sun, J., Zhao, W., Cao, J., Wang, J., Cheng, H., Zhang, L., Guo, Y., Xu, R.: Lips: Large-scale humanoid robot reinforcement learning with parallel-series structures. *arXiv preprint* (2025)